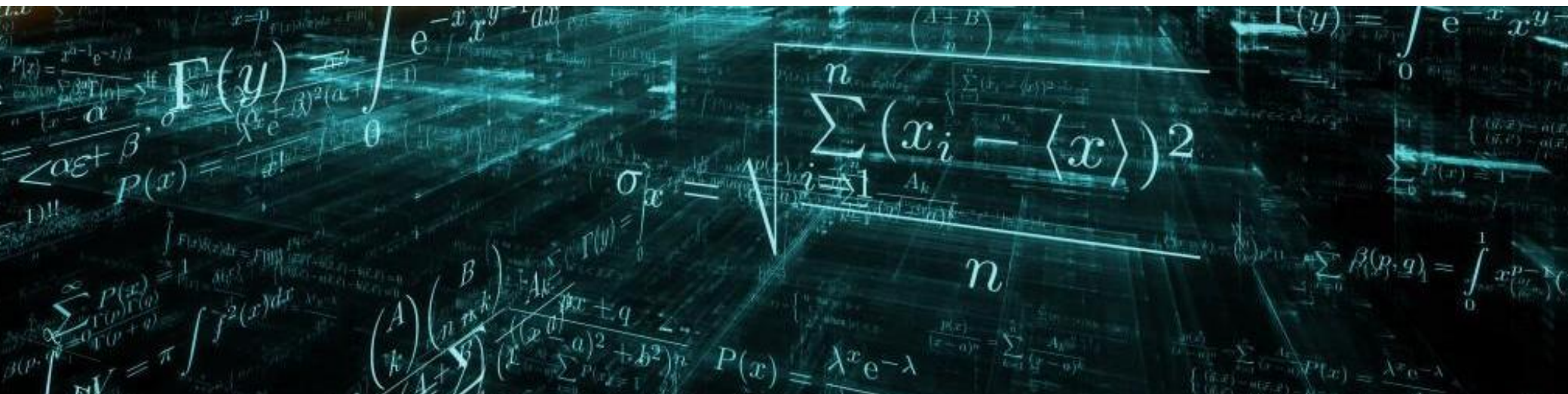


Multimedia Retrieval

Chapter 3: Advanced Text Processing

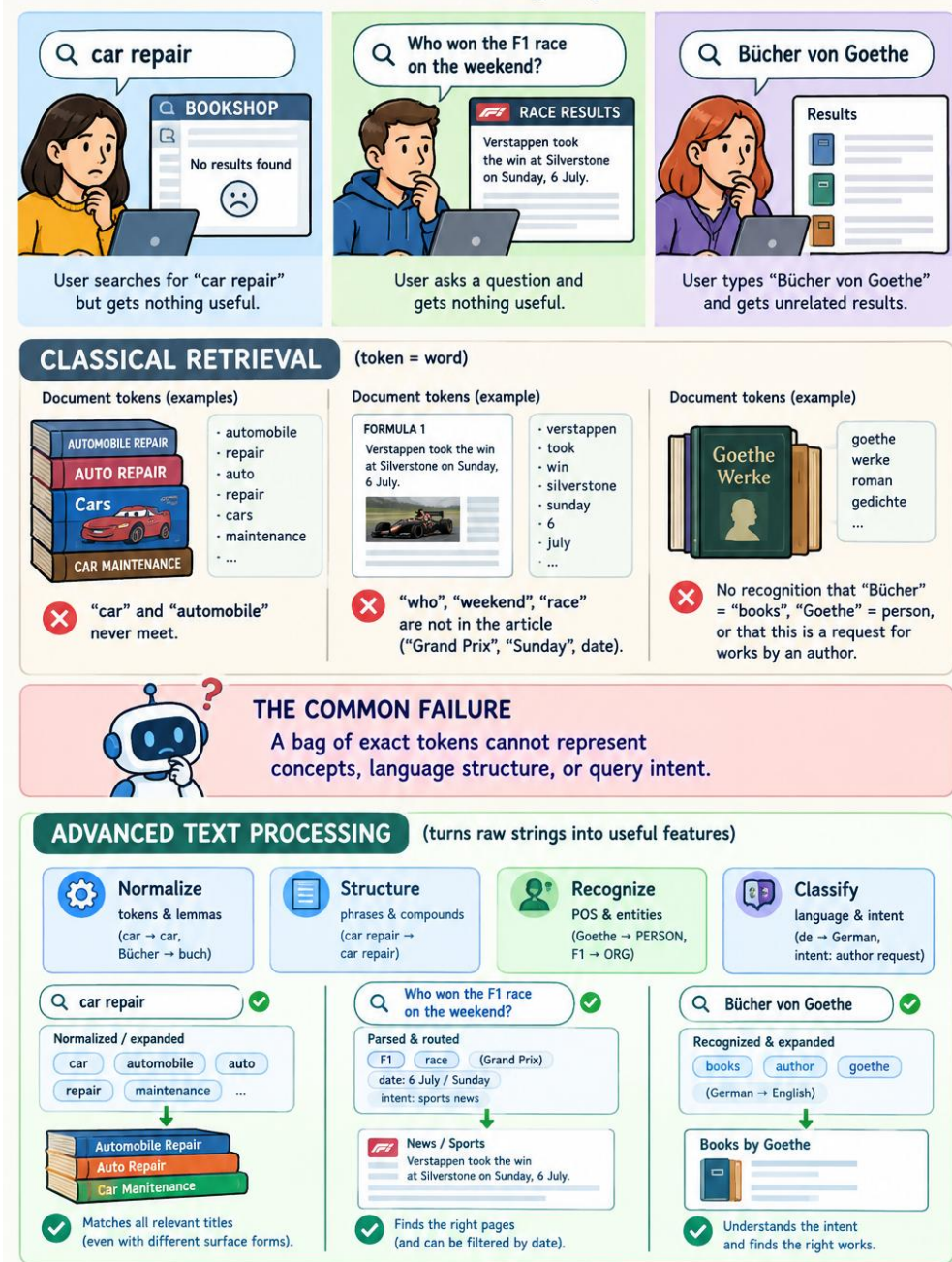
Dr. Roger Weber, roger.weber@gmail.com

Three queries, three failures
Tokenization and Normalization
Stemming and Lemmatization
Phrases and Compounds
Query Understanding
Intent Routing and Classification



Three queries, three failures

- Exact tokens capture surface form, not meaning or structure
- Advanced text processing
 - Normalize → Structure → Recognize → Classify
- Retrieval system
 - recall + routing + filtering + ranking
 - increasing recall can lower precision
- This chapter covers classical approaches for text processing applied to text retrieval
 - Later: embeddings and language models address the remaining gap



Tokenization and Normalization

- Naive tokenizer easily breaks, need more robust way
 - nltk, spaCy tokenizers
 - retrieval cleanup: short tokens, numbers, punctuation, possessive forms
 - case, Unicode, and accents

→ goal: increase recall by collapsing surface forms to same token; this often comes at a precision penalty
- Sentence boundaries important for chunking

▶ [Demo: ch03-01-tokenization.ipynb](#)

Special cases

- Word boundaries can be unclear in scriptio continua languages like Chinese, Japanese, Thai, ancient Greek, and ancient Latin
 - Example: Chinese text requires segmentation to identify words and meaning
 - Programming uses compound literals (QueryParser, word_tokenize) without spaces; assistants split them into meaningful tokens
 - Speech transcription faces continuous phoneme streams without natural breaks

莎拉波娃现在居住在美国东南部的佛罗里达。

莎拉波娃 现在 居住 在 美国 东南部 的 佛罗里达

Sharapova now lives in US southeastern Florida

QueryParser, word_tokenize, build_exercise.py

ðɪskɔ:s'ti:tʃɪz ,mʌltɪ'mi:diəɪ'rɪ'tri:vəl.

ðɪs kɔ:s 'ti:tʃɪz ,mʌltɪ'mi:diə rɪ'tri:vəl.

This course teaches multimedia retrieval.

Sub-word Tokenization

- Sub-word tokenization divides phoneme streams into overlapping sequences
 - Stress symbols combine with following phonemes, expanding the symbol vocabulary
 - Queries like “multimedia” converted into phoneme triples for matching
- Method also applies to written text to handle spelling
 - Prefix word-initial sub-sequences with “#” to distinguish positions (e.g., #tea, #mul)
 - Handles misspellings and flexed forms

This course teaches multimedia retrieval.

[ðɪskɔ:s'ti:tʃɪz,mʌltɪ'mi:diəri'tri:vəl.]

→ ðɪs ɪsk skɔ kɔ:s ɔ:s't ... ,mʌl ʌlt ltɪ tɪ'm ɪ'mi ...

multimedia [,mʌltɪ'mi:diə]

→ ,mʌl ʌlt ltɪ tɪ'm ɪ'mi 'mi:d i:di diə

This course teaches multimedia retrieval.

→ #Thi his #cou our urs rse **#tea eac ach** che hes
#mul ult lti **tim ime med edi dia** ...

Q = “teach multtimedia”

→ **#tea eac ach #mul ult** ltt tti **tim ime med edi dia**

Stemming and Lemmatization

- Stop words – keep them but low weights
- Stemming: reduce surface forms to same root
 - Porter, Lancaster, Snowball
- Lemmatization: reduce surface form to linguistic root (lemma)
 - Wordnet, spaCy

English: go, goes, went, going, house, houses, fly, flew

German: gehen, gehst, ging, gegangen, Haus, Häuser, ...

► [Demo: ch03-02-stemming.ipynb](#)

Stemming: Porter Algorithm

- Porter Algorithm finds a near-stem of words in English

- Porter defines **v** as a „vocal“ if

- it is an **A, E, I, O, U**
- it is a **Y** and the preceding character is not a „vocal“ (e.g. RY, BY)

- All other characters are consonants (**c**)

- Let **C** be a sequence of consonants, and let **V** be a sequence of vocals

- Each word follows the following pattern:

[C](VC)^m[V] **m** is the measure of the word

- further:

- ***o**: stem ends with **cvc**; second consonant must not be W, X or Y (-WIL, -HOP)
- ***d**: stem with double consonant (-TT, -SS)
- ***v***: stem contains a vocal

Rule

Examples

Step 1

a)	SS	-> SS
	IES	-> I
	SS	-> SS
	S	->

caresses	-> caress
ponies	-> poni
caress	-> caress
cats	-> cat

b)	(m>0) EED	->EE
	(*v*) ED	->
	(*v*) ING	->

feed	-> feed
plastered	-> plaster
motoring	-> motor

... +5 more rules if 2nd/3rd rule match

c)	(*v*) Y	->I
----	---------	-----

pony	-> poni
------	---------

Step 2

(m>0) ATIONAL	-> ATE
(m>0) TIONAL	-> TION
(m>0) ENCI	-> ENCE
(m>0) IZER	-> IZE

relational	-> relate
conditional	-> condition
valenci	-> valence
digitizer	-> digitize

... +16 more rules

Rule

Examples

Step 3

(m>0) ICATE -> IC
 (m>0) ATIVE ->
 (m>0) ALIZE -> AL

... +4 more rules

triplicate -> triplic
 formative -> form
 formalize -> formal

Step 4

(m>1) and (*S or *T)ION ->
 (m>1) OU ->
 (m>1) ISM ->

... +16 more rules

adoption -> adopt
 homologou -> homolog
 platonism -> platon

Step 5

a) (m>1) E ->
 (m=1) and (not *O)E ->
 b) (m>1 and *D and *L) -> single letter

rate -> rate
 cease -> ceas
 controll -> control

Lemmatization: WordNet

- The dictionary based algorithms works as follows:
 1. Retrieve POS tag considering context during tokenization
 2. Check dictionary: if word found, return as stem
 3. Check exception list for POS tag: return listed stem if found
 4. Apply POS-based rules to shorten regular inflections, verify in dictionary
 5. If no match, return word as its own stem (names, misspellings, loanwords)

Type	Suffix	Ending
NOUN	s	
NOUN	ses	s
NOUN	xes	x
NOUN	zes	z
NOUN	ches	ch
NOUN	shes	sh
NOUN	men	man
NOUN	ies	y
VERB	s	
VERB	ies	y
VERB	es	e
VERB	es	
VERB	ed	e
VERB	ed	
VERB	ing	e
VERB	ing	
ADJ	er	
ADJ	est	
ADJ	er	e
ADJ	est	e

adj.exc (1500):

...
stagiest stagy
stalkier stalky
stalkiest stalky
stapler stapler
starchier starchy
starchiest starchy
starer starer
starest starest
starrier starry
starriest starry
statelier stately
stateliest stately
...

verb.exc (2400):

...
ate eat
atrophied atrophy
averred aver
averring aver
awoke awake
awoken awake
babied baby
baby-sat baby-sit
baby-sitting baby-sit
back-pedalled back-pedal
back-peddalling back-pedal
backbit backbite
...

noun.exc (2000):

...
neuromata neuroma
neuroptera neuropteron
neuroses neurosis
nevi nevus
nibelungen nibelung
nidi nidus
nielli niello
nilgai nilgai
nimbi nimbus
nimbostrati nimbostratus
noctilucae noctiluca
...

- nltk and spaCy offer multiple stemmers for different languages

– English Example:

- English stems vary by algorithm
- Snowball and Porter yield similar stems
- Lancaster aggressively removes suffixes

term	pos	porter	lancaster	snowball	wordnet	spaCy
blue	ADJ	blue	blu	blue	blue	blue
bottles	NOUN	bottl	bottl	bottl	bottle	bottle
bristled	VERB	bristl	bristl	bristl	bristle	bristle
companion	NOUN	companion	comp	companion	companion	companion
cry	NOUN	cri	cry	cri	cry	cry
discovered	VERB	discov	discov	discov	discover	discover
distant	ADJ	distant	dist	distant	distant	distant
feet	NOUN	feet	feet	feet	foot	foot
flickering	NOUN	flicker	flick	flicker	flickering	flicker
found	ADP	found	found	found	found	find
had	VERB	had	had	had	have	have
have	VERB	have	hav	have	have	have
his	PRON	hi	his	his	his	his
is	VERB	is	is	is	be	be
lofty	ADJ	lofti	lofty	lofti	lofty	lofty
nothing	NOUN	noth	noth	noth	nothing	nothing
one	NUM	one	on	one	one	one
only	ADV	onli	on	onli	only	only
over	ADP	over	ov	over	over	over

```
import nltk
from nltk.corpus import wordnet
import spacy

# building a stemmer
porter = nltk.PorterStemmer()
lancaster = nltk.LancasterStemmer()
snowball = nltk.SnowballStemmer("english")
wordnet = nltk.WordNetLemmatizer()
spacy = spacy.load('en_core_web_sm')

# applying it
porter.stem('discovered')
lancaster.stem('discovered')
snowball.stem('discovered')
wordnet.lemmatize('discovered', 'v')

# spacy processes full text sequence,
# not just one word
for token in spacy('I have discovered it'):
    print(token.text, token.lemma_)
```

– German Example:

- For German, Snowball is rule-based; spaCy is dictionary-based
- Snowball normalizes special characters and lowercase; spaCy handles true linguistic stems
- SpaCy better matches strongly inflected variants for accurate retrieval

term	pos	snowball	spaCy
abgefunden	VERB	abgefund	abfinden
abraten	VERB	abrat	abraten
alleiniger	ADJ	allein	alleinig
alles	PRON	all	alle
am	ADP	am	an
Andererseits	ADV	andererseit	andererseits
Anteil	NOUN	anteil	Anteil
aufgerichtet	VERB	aufgerichtet	aufrichten
Augen	NOUN	aug	Auge
bedeutet	VERB	bedeutet	bedeuten
beschloß	VERB	beschloss	beschließen
blickte	VERB	blickt	blicken
damals	ADV	damal	damals
das	PRON	das	der
dem	PRON	dem	der
den	DET	den	der
des	DET	des	der
Die	DET	die	der
Einerseits	ADV	einerseit	einerseits
endgültig	ADV	endgult	endgültig
energischen	ADJ	energ	energisch
Erbe	NOUN	erb	Erbe
Firma	NOUN	firma	Firma
Flamme	NOUN	flamm	Flamme

– French Example:

- Snowball and spaCy work well for French stemming
- Snowball retains accents, converts words to lowercase
- spaCy preserves casing for names
- Rule-based Snowball vs. dictionary-based spaCy shows similar differences as in German
- SpaCy better maps inflected forms to the same stem
- Snowball may assign different stems to the same root forms

term	pos	snowball	spaCy
Aiguillon	PROPN	aiguillon	Aiguillon
Aramis	X	aram	Aramis
Artagnan	PROPN	artagnan	Artagnan
Tous	ADJ	tous	tout
Tréville	PROPN	trévill	Tréville
agréable	ADJ	agréabl	agréable
aperçu	ADJ	aperçu	apercevoir
aperçut	VERB	aperçut	apercevoir
approcha	PROPN	approch	approcher
arrivé	VERB	arriv	arriver
avaient	AUX	avaient	avoir
avait	AUX	avait	avoir
causant	VERB	caus	causer
cet	DET	cet	ce
comme	CONJ	comm	comme
conciliation	NOUN	concili	conciliation
contraire	NOUN	contrair	contraire
conversation	NOUN	convers	conversation
courtoisie	NOUN	courtois	courtoisie
côté	NOUN	côt	côté
devant	ADP	dev	devant
du	ADP	du	de
emporté	ADJ	emport	emporté
entier	ADJ	enti	entier

N-Grams – frequent pairs

- **N-grams** combine multiple words into single tokens; carry different meaning than sum of its parts
 - mother tongue, red handed, butterfly effect, black box, cold shoulder, silver bullet, piece of cake
 - thai food, prime minister, middle management, crystal clear, chief of staff, speed dial, multimedia retrieval
 - New York City, Salt Lake City, Albert Einstein, Amazon Web Services, Ford Mustang, University of Basel
- Naive approach to identify N-grams in collection
 - count all pairs in collection (sentence boundaries!)
 - most frequent pairs are candidates for new tokens
 - Fix: eliminate pairs that contain stop words

bi-gram	$tf(t_1)$	$tf(t_2)$	$tf(t_1, t_2)$
of the	1198	2323	297
in the	669	2323	186
to the	1071	2323	135
to be	1071	246	96
and the	1313	2323	89
upon the	195	2323	88
I have	936	276	81
that I	614	936	76

↓
remove pairs with
stop words

bi-gram	$tf(t_1)$	$tf(t_2)$	$tf(t_1, t_2)$
Sherlock Holmes	48	94	48
Jefferson Hope	37	42	34
John Ferrier	31	58	26
Brixton Road	15	13	13
said Holmes	207	94	12
Lucy Ferrier	29	58	10
Enoch Drebbler	9	62	9
Salt Lake	9	9	9
could see	96	56	9
young man	40	154	9

► [Demo: ch03-03-phrases-and-compounds.ipynb](#)

Pointwise Mutual Information (PMI)

- Use bi-grams where terms occur exclusively together:

$$pmi(t_1, t_2) = \log_2 \frac{p(t_1, t_2)}{p(t_1) \cdot p(t_2)} = \log_2 \frac{N \cdot tf(t_1, t_2)}{tf(t_1) \cdot tf(t_2)}$$

- Rare bi-grams like “Army Medical” score highest due to low occurrence count; PMI is maximal when

$$tf(t_1) = tf(t_2) = tf(t_1, t_2) = 1$$

- Fix: require minimum frequency (filter rare pairs)

remove rare pairs

bi-gram	$tf(t_1)$	$tf(t_2)$	$tf(t_1, t_2)$	PMI
Army Medical	1	1	1	15.39
Assistant Surgeon	1	1	1	15.39
Avenging Angels	1	1	1	15.39
Beautiful beautiful	1	1	1	15.39
Boarding Establishment	1	1	1	15.39
CITY Died	1	1	1	15.39
Conan Doyle	1	1	1	15.39
Continental Governments	1	1	1	15.39
Copernican Theory	1	1	1	15.39
Cremona fiddles	1	1	1	15.39
Criterion Bar	1	1	1	15.39
Danite Band	1	1	1	15.39

bi-gram	$tf(t_1)$	$tf(t_2)$	$tf(t_1, t_2)$	PMI
Private Hotel	5	5	5	13.07
Scotland Yard	8	6	6	12.39
Salt Lake	9	9	9	12.22
Baker Street	6	11	6	11.93
Lake City	9	13	8	11.52
Brixton Road	15	13	13	11.48
Never mind	5	37	5	10.18
Jefferson Hope	37	42	34	9.87
Enoch Drebber	9	62	9	9.43
old farmer	38	9	5	9.29
John Ferrier	31	58	26	9.28
Joseph Stangerson	13	43	7	9.07
young hunter	40	14	6	8.84
Sherlock Holmes	48	94	48	8.83
two detectives	75	9	5	8.31
Lucy Ferrier	29	58	10	7.99

Likelihood Ratio (LHR)

- LHR tests two hypothesis:
 - $L(H_1)$: words in pair are dependent
 - $L(H_1)$: words in pair are independent
 - log-likelihood ratio

$$\log \lambda = \log \frac{L(H_1)}{L(H_2)}$$

- Stop words reappear because LHR captures strong dependencies, e.g., “I am”
- Fix: eliminate pairs that contain stop words

remove pairs with stop words

bi-gram	$tf(t_1)$	$tf(t_2)$	$tf(t_1, t_2)$	LHR
Sherlock Holmes	48	94	48	618.26
of the	1198	2323	297	517.86
Jefferson Hope	37	42	34	491.94
don t	32	88	32	409.61
to be	1071	246	96	395.47
had been	470	147	63	378.63
in the	669	2323	186	359.01
he said	630	207	72	352.64
John Ferrier	31	58	26	330.18
I have	936	276	81	300.87
I am	936	41	39	284.24
upon the	195	2323	88	259.81

bi-gram	$tf(t_1)$	$tf(t_2)$	$tf(t_1, t_2)$	LHR
Sherlock Holmes	48	94	48	618.26
Jefferson Hope	37	42	34	491.94
John Ferrier	31	58	26	330.18
Brixton Road	15	13	13	224.92
Salt Lake	9	9	9	170.49
Lake City	9	13	8	129.82
Enoch Drebber	9	62	9	119.12
Scotland Yard	8	6	6	109.52
Baker Street	6	11	6	103.36
Private Hotel	5	5	5	100.59
Lucy Ferrier	29	58	10	96.68
Lauriston Gardens	4	4	4	82.26
Joseph Stangerson	13	43	7	79.97
Never mind	5	37	5	71.28
little girl	80	27	8	68.66
young hunter	40	14	6	65.60

Splitting (German) Compounds

- Compounds combine two or more base words, sometimes with binding syllables or characters
 - Most languages allow basic compounds; German and Finnish allow arbitrarily long compounds
- Finnish:
 - *kolmivaihekilowattituntimittari* **en:** electricity meter
 - *atomiydinenergiareaktorigeneraattorilauhduttajaturbiiniratasvaihde*
en: atomic nuclear energy reactor generator condenser turbine cogwheel stage
- German:
 - *Wolkenkratzer* **en:** skyscraper
 - *Rinderkennzeichnungs- und Rindfleischetikettierungsüberwachungsaufgabenübertragungsgesetz*
(German (law in Mecklenburg-Vorpommern, 1999-2013))
en: cattle marking and beef labeling supervision duties delegation law
- Endocentric: meaning from constituent parts, head defines semantics (e.g., ‘sunglasses’)
- Exocentric: meaning not from parts, head absent, meaning differs from constituents (e.g., ‘skyscraper’)

Automatic Splitting with Hyphenation Rules

- Automatic splitting uses rule-based or morphological analysis, varies by language
 - English: split on hyphens or syllables ('must-have' → 'must', 'have')
 - German: split on syllables and handle binding letters ('Schiffahrtsskapitän' → 'Schiff', 'fahrt', ...)
- Generate all possible combinations of splits and choose split with highest score

wolkenkratzer

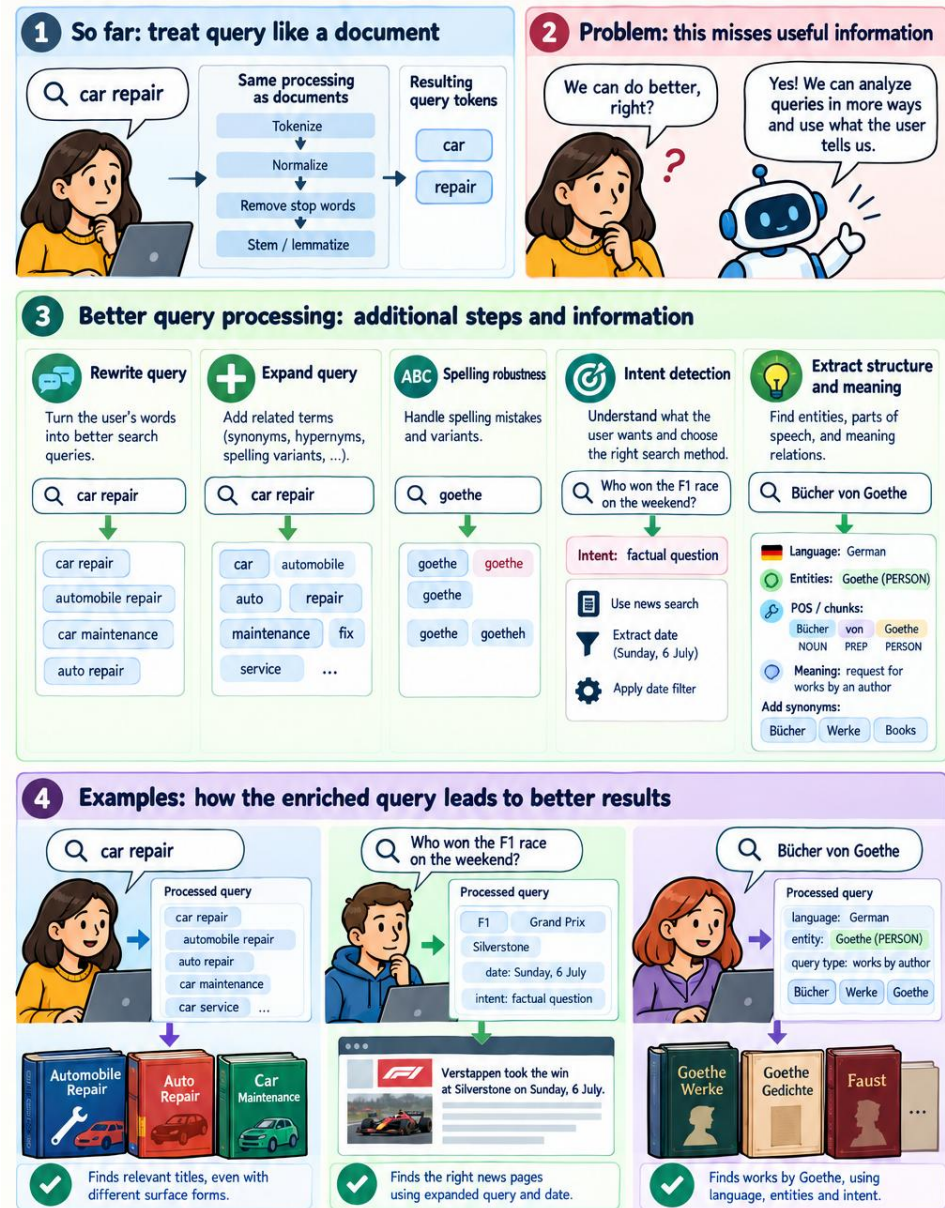
- $S_1 = (\text{wol}, \text{ken}, \text{krat}, \text{zer})$
- $S_2 = (\text{wolken}, \text{krat}, \text{zer})$
- $S_3 = (\text{wol}, \text{kenkrat}, \text{zer})$
- $S_4 = (\text{wol}, \text{ken}, \text{kratzer})$

$$\text{score}(S) = \frac{1}{|S|} \sum_{p_i \in S} \log \frac{tf(p_i)}{N}$$

- Scoring: use German word-frequency table (wordfreq package, offline)
 - returns a Zipf value: higher = more common
 - 0 = word never appears

Query Understanding

- So far: query processed similar to document
- How can we improve precision / recall further
 - rewrite query
 - expand query
 - spelling robustness
 - intent detection (use better search methods)
- Extract additional information
 - synonyms, homonyms, hypernyms, ...
 - POS, NER and chunking
 - spelling variants (or spelling mistakes)
 - phonetic matching (e.g., for names)



► Demo: [ch03-04-query-understanding.ipynb](#)

End-to-End Processing flow

- Language detection
 - Stopword counting
 - Naive Bayes on sub-word
 - (dictionary based)
- Intent detection
 - query: "who" + person (NER)
→ use optimized people search
 - query: "bücher von Goethe"
→ use optimized book search with author
 - Classical approach: Naive Bayes over words in query
 - Modern approach: (small) language model extracts intent
 - Hybrid approach: Naive Bayes first, if not confident then with language model

Stage	Output
Tokenize	['Bücher', 'von', 'Goethe']
Normalize (case)	['bücher', 'von', 'goethe']
Detect language	German (confidence 0.97)
Stop-word filter	drop ['von'] -> ['bücher', 'goethe']
Stem	['buch', 'goeth']
POS tag	[('Bücher', 'NOUN'), ('von', 'ADP'), ('Goethe', 'PROPN')]
Named entities	[('Goethe', 'PER')]
Classify intent	book_search

► [Demo: ch03-05-intent-routing.ipynb](#)

Naive Bayes

- What we need for language and intent detection
 - Feature vector $\mathbf{x}$, class C_k , prior, likelihood, evidence, posterior
 - Often assume independent features x_i
 - Prediction: Maximum A Posteriori (MAP)
- Estimation of prior
 - Count class occurrences in training data
 - ...or assume equal distribution of classes
 - Note: prevalence of classes can negatively impact predictions

$$P(C_k|\mathbf{x}) = \frac{P(\mathbf{x}|C_k) \cdot P(C_k)}{P(\mathbf{x})}$$

$$\text{posterior} = \frac{\text{likelihood} \cdot \text{prior}}{\text{evidence}}$$

$$k^* = \underset{k}{\operatorname{argmax}} P(C_k) \cdot \prod_{j=1}^M P(x_j|C_k)$$

That is it! The equation describes the decision rule of Naïve Bayes. The only thing left are the estimates for the probabilities on the right hand side

$$P(C_k) = \frac{N_k}{N} \qquad \text{uniform: } P(C_k) = \frac{1}{K}$$

Likelihood Estimation

- Model feature distribution in classes from training data (maximum likelihood estimator)
 - Bernoulli distribution: for set-of-words, e.g., intent detection based on presence / absence of keywords
 - smoothing: prevent $p_{k,j} = 0$ for features not seen in a class during training
 - Multinomial distribution for bag-of-words, e.g., language detection on sub-words
 - smoothing: prevent $p_{k,j} = 0$ for features not seen in a class during training

$$p_{k,j} = \frac{N_k(x_j = 1)}{N_k}$$

or smoothed:

$$p_{k,j} = \frac{\min(N_k - 1, \max(1, N_k(x_j = 1)))}{N_k}$$

$$p_{k,j} = \frac{n_{k,j}}{\sum_l n_{k,l}}$$

or smoothed:

$$p_{k,j} = \frac{n_{k,j} + 1}{\sum_l n_{k,l} + M}$$

Key Takeaways

- Classical retrieval fails two ways; this chapter closes both
 - recall gap on document side
 - intent gap on query side
- Tokenization is the first stage and the biggest source of quality issues
- Stemming = fast pseudo-stem; lemmatization = real base form
- PMI (strength) and LHR (evidence) rank the same phrases differently
- Only endocentric compounds split safely
- Query understanding = language + POS + NER + spelling → routing
- One Naive Bayes model powers both language detection and intent routing

Further Reading

- Porter, M. F. (1980). **An Algorithm for Suffix Stripping**. *Program*, 14(3), 130-137. [Author copy](#). The original Porter Stemmer paper, still the reference for how to build a rule-based English stemmer and the ancestor of every rule-based stemmer that followed.
- Paice, C. D. (1990). **Another Stemmer**. *ACM SIGIR Forum*, 24(3), 56-61. Introduced the Lancaster stemmer and the aggressive suffix-stripping strategy that trades precision for speed.
- Manning, C. D., & Schütze, H. (1999). **Foundations of Statistical Natural Language Processing**, Chapter 5: Collocations. *MIT Press*. The standard reference for PMI, likelihood-ratio testing, and other statistical collocation-scoring methods; source of the LHR formulation used in this chapter.
- Kiss, T., & Strunk, J. (2006). **Unsupervised Multilingual Sentence Boundary Detection**. *Computational Linguistics*, 32(4), 485-525. [Free PDF](#). The Punkt algorithm, still the sentence-segmentation default in NLTK almost two decades later.
- Koehn, P., & Knight, K. (2003). **Empirical Methods for Compound Splitting**. *Proceedings of the 10th Conference of the European Chapter of the ACL*, 187-193. [Free PDF](#). The frequency-based compound-splitting method this chapter uses, evaluated for machine translation but applicable directly to retrieval.
- Minixhofer, B., Pfeiffer, J., & Vulić, I. (2023). **CompoundPiece: Evaluating and Improving Decompounding Performance of Language Models**. [arXiv:2305.14214](#). Shows that modern subword tokenizers (SentencePiece, BPE) still decompound German poorly, motivating why explicit decompounders remain in hybrid retrieval stacks.

Implementations and tools

- [NLTK](#) — Python toolkit for classical NLP; ships all the stemmers, tokenizers, POS taggers, and collocation finders used in this chapter.
- [spaCy](#) — Industrial-strength NLP library with neural taggers and lemmatizers for many languages.
- [WordNet](#) — Lexical database for English synonyms, hypernyms, and hyponyms.
- [Snowball](#) — Multi-language stemmer framework, source of the German and French stemmers used in section 2.
- [lingua-language-detector](#) — Character-n-gram Naive Bayes language detector covering 70+ languages.
- [Elasticsearch decompounder documentation](#) — Production compound-splitter configuration for German, Dutch, and other compounding languages.